SMART INDIA HACKATHON 2026

GeM-Intel

Advanced Price Intelligence & Procurement Audit Engine

System Architecture & Technical Design Report

Problem Statement ID	SIH 1360
Problem Statement Title	Price Comparison of GeM Products with Other e-Marketplaces
Theme	Smart Automation
PS Category	Software
Team Name	Team Valeryon
Document Type	Build / Implementation Report
Date	August 2026

Table of Contents

1. Introduction & Problem Statement

1.1 Background

The Government e-Marketplace (GeM) is India's dedicated online procurement platform, used by central and state government departments, autonomous bodies and public-sector undertakings (PSUs) to buy goods and services. GeM was created to bring transparency, efficiency and speed to public procurement, replacing manual tendering with an online catalog-and-bid model. However, GeM operates as a largely closed pricing ecosystem: officers raising a purchase order have no fast, reliable way to check whether a listed price is fair when compared against identical or equivalent products available on open-market platforms such as Amazon, Flipkart or IndiaMART.

General Financial Rules (GFR), 2017 — specifically Rule 149 — require procuring authorities to satisfy themselves of the "Reasonableness of Rates" before a purchase is approved. In practice this compliance step is done manually: an officer opens multiple marketplace tabs, eyeballs prices, and creates an ad-hoc note as evidence. This process is slow, inconsistent across officers, easy to skip under workload pressure, and leaves no verifiable audit trail. It also creates room for inflated or cartelized listings to go unchallenged.

1.2 Problem Statement (SIH 1360)

Develop a cost/price comparison solution that benchmarks the prices of products listed on GeM against prices for the same or equivalent products on other e-marketplaces and e-commerce platforms, using data scraping, APIs and analytics, so that government buyers, auditors and the public can verify that GeM prices are fair and competitive.

1.3 Objectives

- Provide procurement officers a fast, one-click way to benchmark any GeM product's price against comparable listings on Amazon, Flipkart and IndiaMART.
- Normalize prices to a true landed cost (GST, freight, AMC/warranty) rather than comparing misleading headline prices.
- Automatically match products across platforms even when titles, brands or specification formats differ.
- Generate a tamper-proof, independently verifiable audit certificate as documentary evidence of GFR Rule 149 compliance.
- Proactively detect anomalous pricing patterns that may indicate cartelization or price gouging, before a purchase order is raised.

2. Proposed Solution — GeM-Intel Overview

2.1 Solution Summary

GeM-Intel is an AI-powered price-intelligence and procurement-audit platform that benchmarks GeM listings against Amazon, Flipkart and IndiaMART in near real time. It is built around four cooperating modules — Semantic Matching, TCO (Total Cost of Ownership) Normalization, Audit Certificate generation, and Anomaly Detection — that together turn a manual, error-prone compliance chore into an automated, defensible, one-click workflow.

2.2 Core Modules

Module	Function	Key Technique
SM — Semantic Matching	Matches the same underlying product across platforms despite differing titles/specs (e.g. “HP ProBook i5” vs “Hewlett Packard 440 G8 Core i5”).	Vector embeddings on title + specification text; 95%+ target matching accuracy.
TC — TCO Normalizer	Converts each platform's headline price into a true landed cost so comparisons are apples-to-apples.	PIN-code based GST, freight and AMC/warranty normalization.
RC — Audit Certificate	Produces a documentary, tamper-proof compliance record for every comparison performed.	Blockchain-hashed, digitally signed PDF report.
AD — Anomaly Detection	Flags GeM listings priced significantly above the fair market value derived from other platforms.	ARIMA / Prophet time-series forecasting; flags 20%+ overpricing.

2.3 How It Addresses the Problem

- **Cross-platform product identity resolution —** resolves inconsistent product titles and specification formats through semantic, spec-based matching instead of literal keyword search, so a product isn't missed just because its name differs by platform.
- **Fair, landed-cost comparison —** eliminates unfair raw-price comparisons by normalizing for freight, GST and AMC, ensuring GeM prices are judged against a true landed cost rather than a misleading headline price.
- **Ready compliance evidence —** gives procurement officers a ready, tamper-proof audit certificate as documented proof that market rates were checked — directly satisfying GFR compliance requirements.
- **Proactive fraud/overpricing detection —** flags anomalous or inflated GeM listings, surfacing suspected cartelization or price gouging before a purchase is made, not after the fact.

2.4 Innovation and Uniqueness

- **Spec-aware semantic matching —** vector embeddings on titles and specifications, instead of the simple keyword/text matching used by conventional price-comparison tools.

- **Dynamic landed-cost normalization —** PIN-code based GST, freight and warranty standardization for a genuine apples-to-apples comparison rather than a headline-price-only view.
- **Tamper-proof audit trail —** blockchain-hashed audit certificates make the compliance record independently verifiable — a first for GeM-linked procurement audits.
- **Predictive, not reactive, anomaly detection —** ARIMA/Prophet forecasting proactively flags cartelization and price gouging rather than only reporting current prices after the fact.

3. System Architecture

3.1 Architectural Style & Principles

GeM-Intel follows a layered, service-oriented architecture on a MERN + Python hybrid stack. Each tier is independently deployable and horizontally scalable, communicating over REST and an internal message queue rather than direct function calls, so that slow or unreliable operations (web scraping, PDF generation, model inference) never block the user-facing API.

- Separation of concerns: presentation (React/Chrome Extension), orchestration (Node/Express), intelligence (Python/FastAPI) and persistence (MongoDB/Redis) are cleanly separated tiers.
- Asynchronous-first: any operation whose latency depends on a third-party website (scraping) or heavy computation (NLP inference, PDF rendering) is queued and processed by a worker, never executed inline in an HTTP request.
- Cache-aside pattern: Redis sits in front of MongoDB for hot price lookups so repeat comparisons on the same product are served in milliseconds rather than re-scraped.
- Stateless API layer: the Express gateway holds no session state itself (JWT-based auth), so additional API instances can be added behind a load balancer without sticky sessions.
- Defense in depth for scraping: rotating proxies, DaaS providers and headless browser fingerprint randomization reduce the risk of IP blocks disrupting data collection.

3.2 High-Level Architecture Diagram

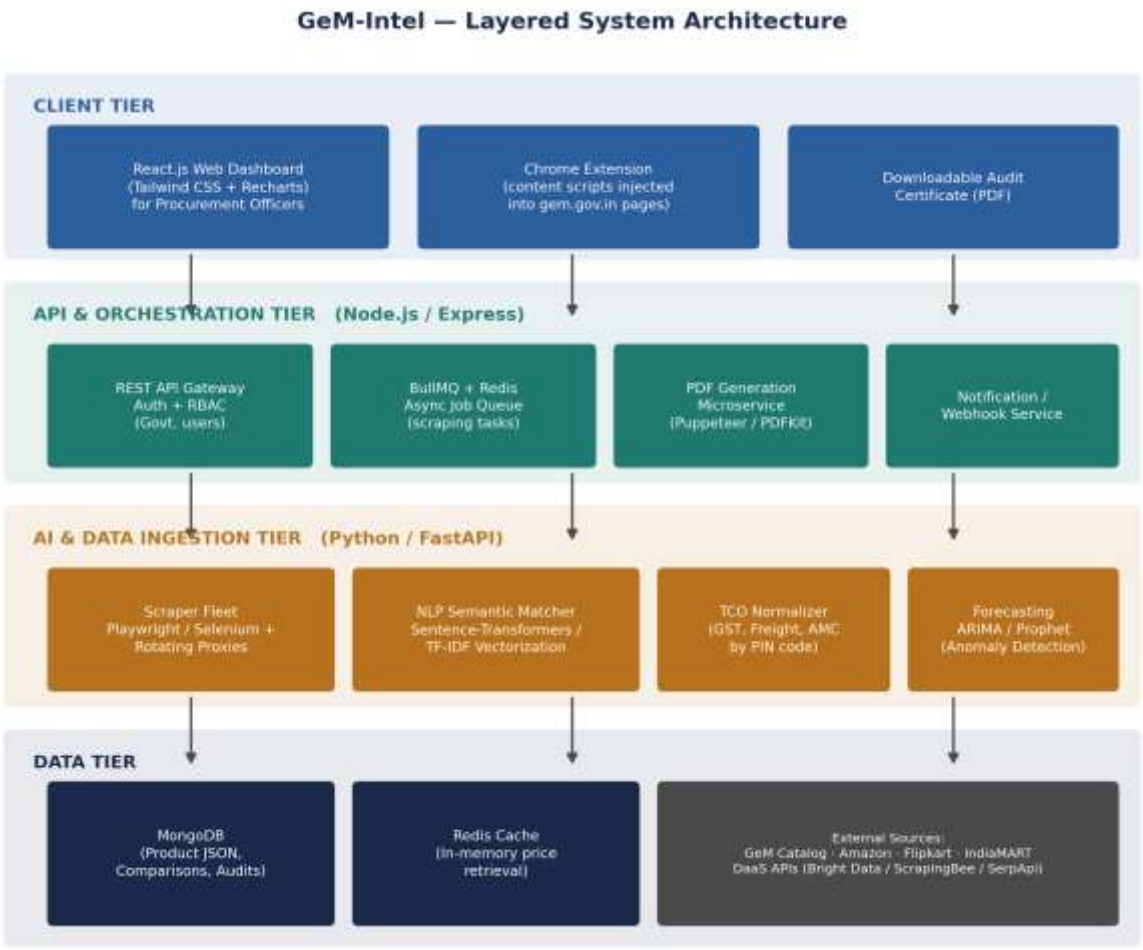

Figure 1: GeM-Intel layered system architecture — Client, API/Orchestration, AI/Data Ingestion and Data tiers.

3.3 Tier-by-Tier Description

3.3.1 Client Tier

- React.js Web Dashboard — the primary interface for procurement officers, styled with Tailwind CSS; uses Recharts for price-comparison and trend visualizations.
- Chrome Extension — injects content scripts directly into gem.gov.in product pages, auto-detecting the product being viewed and offering an inline “Compare Price” action without the officer leaving the GeM portal.
- Audit Certificate Viewer/Downloader — renders and lets officers download the generated PDF compliance certificate directly from the dashboard or extension.

3.3.2 API & Orchestration Tier (Node.js / Express)

- REST API Gateway — single entry point for all client requests; handles authentication (JWT) and Role-Based Access Control (RBAC) for government users (Officer, Auditor, Admin roles).
- BullMQ + Redis Job Queue — decouples request handling from long-running scraping/matching jobs; each comparison request becomes a tracked, retryable job.

- PDF Generation Microservice — a dedicated Node service (Puppeteer/PDFKit) that renders the audit certificate as a print-ready, digitally-stamped PDF.
- Notification/Webhook Service — pushes job-completion events to the dashboard (WebSocket/SSE) and optionally to departmental email/webhook endpoints.

3.3.3 AI & Data Ingestion Tier (Python / FastAPI)

- Scraper Fleet — headless browsers (Playwright/Selenium) with rotating proxies that extract product title, specifications, seller, price, freight and warranty terms from each target marketplace.
- NLP Semantic Matcher — encodes product titles and specifications using Sentence-Transformers (with TF-IDF as a fast pre-filter) and ranks candidate matches by cosine similarity.
- TCO Normalizer — a rules-and-data engine that converts each platform's quoted price into a landed cost using PIN-code-specific GST slabs, estimated freight and AMC/warranty value.
- Forecasting/Anomaly Engine — ARIMA/Prophet models trained on historical price series per product category to establish an expected price band and flag deviations.

3.3.4 Data Tier

- MongoDB — primary datastore; stores product documents, scrape snapshots, comparison results and audit-certificate metadata as flexible JSON documents (schema varies naturally by product category).
- Redis — in-memory cache for hot product/price lookups and as the backing store for the BullMQ job queue.
- External Sources — the GeM public catalog plus Amazon, Flipkart and IndiaMART (accessed via a mix of direct scraping and commercial DaaS APIs).

4. Detailed Module Design

4.1 Semantic Matching Module (SM)

Purpose: given a GeM product listing, find the equivalent (or closest) listing on each target marketplace, even when brand names, model codes or spec formatting differ.

- Step 1 — Text normalization: lower-casing, unit standardization (e.g. “8GB” vs “8 GB”), brand-alias resolution (“HP” = “Hewlett Packard”) and stopword removal.
- Step 2 — Candidate retrieval: a fast TF-IDF / BM25 pass over the target marketplace's indexed catalog narrows millions of listings down to a shortlist (top-50) sharing key tokens (brand, category, core specs).
- Step 3 — Semantic re-ranking: the shortlist is embedded with a Sentence-Transformers model (e.g. all-MiniLM-L6-v2 or a domain-fine-tuned variant) and ranked by cosine similarity against the GeM listing's embedding.
- Step 4 — Specification cross-check: structured attributes (RAM, processor, storage, warranty period) extracted via regex/NER are compared explicitly; mismatches on hard attributes (e.g. RAM size) demote or disqualify a candidate even if text similarity is high.
- Step 5 — Confidence scoring: a combined score (semantic similarity + attribute match rate) above a tuned threshold is auto-accepted; scores in a middle band are routed to manual officer review; low scores are discarded.

Target accuracy: 95%+ correct top-1 match on tested product categories (as stated in the idea submission), validated against a manually labeled test set before production rollout.

4.2 TCO Normalizer (TC)

Purpose: convert every platform's headline price into a comparable “landed cost” — what the buying department would actually pay, delivered, inclusive of tax and post-sale support.

- GST normalization: applies the correct HSN-code-based GST slab to each platform's price so tax treatment (inclusive vs exclusive pricing) doesn't distort the comparison.
- Freight normalization: estimates delivery cost using the buyer's PIN code, product weight/dimensions and each platform's stated shipping policy (free-above-threshold, flat-rate, or seller-calculated).
- AMC / warranty normalization: converts differing warranty lengths and optional AMC costs into an annualized ownership-cost adjustment so a cheaper item with a shorter warranty isn't unfairly favored.
- Output: a single “Landed Cost” figure per platform per product, plus a breakdown table (base price / GST / freight / AMC) shown transparently to the officer for auditability.

4.3 Audit Certificate Generator (RC)

Purpose: produce a documentary, tamper-evident record proving that a market-rate check was performed, satisfying GFR 2017 Rule 149's “Reasonableness of Rates” requirement.

- Content: GeM product & price, matched listings and landed costs on each comparator platform, variance %, Fair Market Value determination, timestamp, and the officer's identity.
- Rendering: the PDF Generation Microservice (Puppeteer/PDFKit) lays out the certificate as a formatted, printable government document.
- Tamper-proofing: a SHA-256 hash of the certificate's content is computed and stored on an append-only ledger (a private/consortium blockchain or a hash-chained log); the certificate embeds this hash plus a QR code so any party can independently verify the document has not been altered after issuance.
- Verification portal: a lightweight public verification endpoint accepts a certificate ID or QR scan and re-computes/compares the hash to confirm authenticity.

4.4 Anomaly Detection Module (AD)

Purpose: proactively flag GeM listings priced well above the fair market range, surfacing possible cartelization or price gouging before a purchase order is approved.

- Time-series modelling: ARIMA and Facebook Prophet models are fit per product category (or per SKU where sufficient history exists) on historical price series scraped over time.
- Expected-band computation: each model produces a forecasted price band (expected value ± confidence interval) for the current period.
- Flagging rule: a GeM listing priced 20% or more above the forecast band's upper bound (as stated in the idea submission) — or above the landed cost derived from other platforms — is flagged “Review Required” instead of “Compliant.”
- Escalation: repeated flags on the same seller/category feed a simple cartelization heuristic (e.g. multiple sellers converging on the same inflated price) surfaced to auditors for further investigation.

5. Data Flow & Processing Workflow

A single comparison request flows through five stages end-to-end, from trigger to a compliance-ready output. Each stage is handled by a different tier, connected asynchronously through the job queue so that the officer-facing UI stays responsive even while scraping and model inference run in the background.

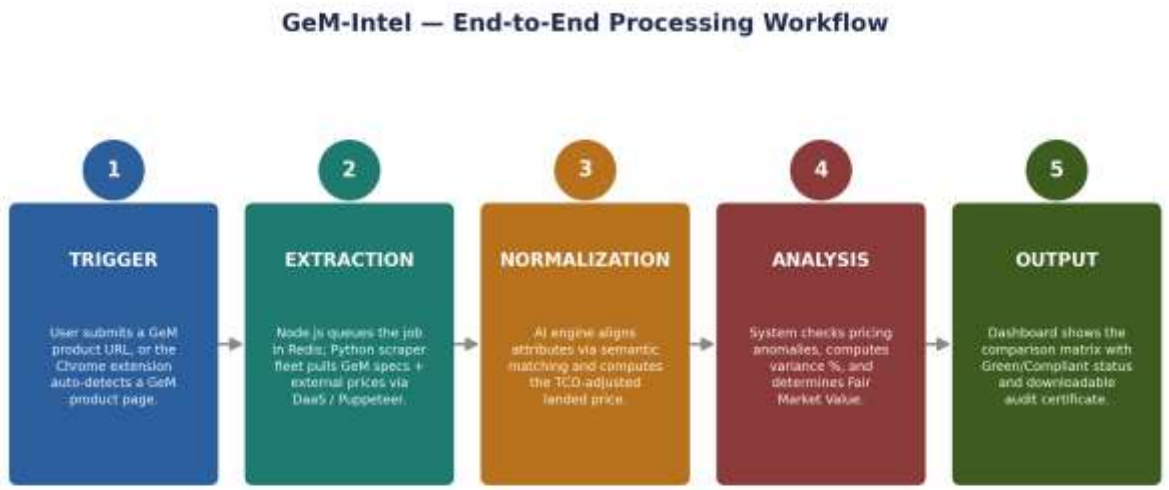

Figure 2: End-to-end request processing workflow.

#	Stage	Owning Tier	Detail
1	Trigger	Client Tier	Officer submits a GeM product URL on the dashboard, or the Chrome extension auto-detects an open GeM product page and offers a one-click compare action.
2	Extraction	API + AI Tier	Node.js enqueues a job in Redis via BullMQ; the Python scraper fleet pulls GeM specifications and concurrently fetches matching listings/prices from Amazon, Flipkart and IndiaMART via DaaS APIs / Puppeteer.
3	Normalization	AI Tier	The semantic matcher aligns product attributes across platforms; the TCO Normalizer computes the PIN-code-adjusted landed price for each matched listing.
4	Analysis	AI Tier	The system computes variance % between GeM's landed price and the Fair Market Value derived from comparator platforms, and runs the anomaly-detection check.
5	Output	API + Client Tier	Results are cached in Redis/MongoDB and pushed to the dashboard as a comparison matrix with a Green (Compliant) / Amber (Review) / Red (Non-compliant) status, plus a downloadable audit certificate.

6. Technology Stack

GeM-Intel uses a MERN + Python hybrid stack: JavaScript/TypeScript for everything user-facing and orchestration-related, and Python for everything data-science-related, connected through REST APIs and a shared Redis-backed job queue.

6.1 Layer-wise Stack

Layer	Technology	Purpose
Frontend — Web	React.js, Tailwind CSS, Recharts	Procurement officer dashboard, comparison matrix and analytics views.
Frontend — Browser Extension	Chrome Extension (Manifest V3, content scripts)	In-context “Compare on GeM-Intel” action injected directly into gem.gov.in.
API Gateway	Node.js, Express.js	Authentication, RBAC, request routing, rate limiting.
Job Queue	BullMQ, Redis	Asynchronous processing of scraping, matching and PDF-generation jobs.
PDF/Certificate Service	Node.js, Puppeteer / PDFKit	Renders the audit certificate as a formatted, print-ready PDF.
AI/ML Service	Python, FastAPI	Hosts the semantic-matching, TCO-normalization and anomaly-detection endpoints.
Scraping Fleet	Playwright, Selenium, rotating proxies	Headless extraction of product data from GeM and comparator marketplaces.
NLP Matching	Sentence-Transformers, TF-IDF (scikit-learn)	Spec-aware semantic product matching across platforms.
Forecasting	ARIMA (statsmodels), Facebook Prophet	Time-series price forecasting and anomaly/overpricing detection.
Primary Database	MongoDB	Flexible JSON storage for products, comparisons and audit metadata.
Cache / Queue Store	Redis	Hot price cache and BullMQ backing store.
Data Sourcing (3rd-party)	Bright Data, ScrapingBee, SerpApi	Proxy rotation and CAPTCHA handling for reliable large-scale scraping.
Containerization	Docker, Docker Compose	Reproducible local and staging environments for every service.
Auth	JWT, bcrypt	Stateless authentication and credential hashing for government users.

7. Data Sourcing Strategy

Because none of the target marketplaces expose a stable, public, price-level API, GeM-Intel uses a hybrid ingestion approach that blends direct scraping with commercial Data-as-a-Service (DaaS) providers, while keeping a path open to official APIs where available.

- **GeM public catalog parsing —** the public gem.gov.in product catalog is parsed directly for baseline specifications, since this is GeM's own authoritative listing data and requires no third-party access.
- **Custom scraping (Puppeteer/Playwright) —** used for platforms/pages where a lightweight, in-house scraper is sufficient and cost-effective for the expected query volume.
- **Third-party DaaS APIs (Bright Data, ScrapingBee, SerpApi) —** used for high-volume or anti-bot-hardened targets, offloading proxy rotation and CAPTCHA solving to a specialized provider rather than building that infrastructure in-house.
- **Official API pathway (future) —** where a marketplace offers a verified-partner or seller API, GeM-Intel is designed to plug that in as a first-class data source once departmental verification/partnership is granted, reducing reliance on scraping over time.

7.1 Data Freshness & Caching

- Every successfully matched price is cached in Redis with a time-to-live (TTL) appropriate to the category's price volatility (e.g. shorter TTL for electronics, longer for stationery).
- A scheduled re-scrape job refreshes high-traffic products ahead of TTL expiry so officers rarely wait on a cold cache.
- Every scrape is stored as an immutable, timestamped snapshot in MongoDB (not overwritten), which is also what feeds the ARIMA/Prophet forecasting models.

8. Database Design

MongoDB's flexible document model suits GeM-Intel well because product specifications vary widely by category (a laptop and a filing cabinet share almost no attributes), and storing them as schema-light JSON avoids constant relational migrations. Redis complements this as a cache and queue store. Indicative collection designs follow.

8.1 MongoDB Collections

users

Field	Type	Description
_id	ObjectId	Unique user identifier.
name, email	String	Officer/auditor identity.
role	Enum	“officer” | “auditor” | “admin” — drives RBAC.
department	String	Government department/PSU the user belongs to.
passwordHash	String	bcrypt-hashed credential.
createdAt	Date	Account creation timestamp.

gem_products

Field	Type	Description
_id	ObjectId	Unique product identifier.
gemUrl	String	Source GeM listing URL.
title, brand, category	String	Raw listing metadata as scraped.
specifications	Object	Free-form key/value spec map (varies by category).
priceHistory	Array<{price, scrapedAt}>	Immutable time-series of observed GeM prices.
lastScrapedAt	Date	Timestamp of the most recent successful scrape.

comparisons

Field	Type	Description
_id	ObjectId	Unique comparison run identifier.
gemProductId	ObjectId (ref)	Link to the gem_products document being benchmarked.
requestedBy	ObjectId (ref)	The officer who triggered the comparison.
pinCode	String	Delivery PIN code used for landed-cost normalization.

Field	Type	Description
matches	Array<Object>	One entry per platform: {platform, matchedListingUrl, similarityScore, basePrice, gst, freight, amc, landedCost}.
fairMarketValue	Number	Computed benchmark value across matched listings.
variancePercent	Number	% difference between GeM landed price and fairMarketValue.
status	Enum	“compliant” | “review_required” | “non_compliant”.
createdAt	Date	Comparison timestamp.

audit_certificates

Field	Type	Description
_id	ObjectId	Unique certificate identifier (also the public verification ID).
comparisonId	ObjectId (ref)	Link to the underlying comparisons document.
pdfUrl	String	Storage location of the generated PDF.
contentHash	String	SHA-256 hash of the certificate content.
ledgerTxId	String	Reference to the blockchain/hash-chain entry anchoring contentHash.
issuedTo	ObjectId (ref)	Officer the certificate was issued to.
issuedAt	Date	Issuance timestamp.

8.2 Redis Key Design

Key Pattern	Type	Purpose
price:{platform}:{productHash}	String/Hash, TTL	Cached landed price for a matched listing, keyed by platform + a hash of normalized product identity.
job:scrape:{jobId}	BullMQ job	Tracks an in-flight scraping/matching job and its status.
session:{userId}	String, TTL	Optional short-lived session/refresh-token metadata.
ratelimit:{userId}:{window}	Counter, TTL	API rate-limiting counters per user per time window.

9. API Design

The Express API gateway exposes a versioned REST interface. All endpoints (except health checks and the public verification endpoint) require a valid JWT and are subject to RBAC checks.

Method & Endpoint	Access	Description
POST /api/v1/auth/login	Public	Authenticates a government user and issues a JWT.
POST /api/v1/compare	Officer	Submits a GeM product URL + PIN code; enqueues a comparison job and returns a jobId.
GET /api/v1/compare/{jobId}	Officer	Polls/streams the status and result of a queued comparison job.
GET /api/v1/products/{id}	Officer, Auditor	Fetches a stored GeM product document and its price history.
GET /api/v1/comparisons	Officer, Auditor	Lists past comparisons, filterable by department, status or date range.
GET /api/v1/comparisons/{id}	Officer, Auditor	Fetches full detail of one comparison, including per-platform matches and variance.
POST /api/v1/certificates/{comparisonId}	Officer	Triggers audit-certificate generation for a completed comparison.
GET /api/v1/certificates/{id}	Officer, Auditor	Downloads the generated PDF certificate.
GET /api/v1/certificates/{id}/verify	Public	Recomputes and compares the content hash to confirm certificate authenticity.
GET /api/v1/anomalies	Auditor, Admin	Lists GeM listings currently flagged by the anomaly-detection module.
GET /api/v1/health	Public	Liveness/readiness probe for orchestration tooling.

10. Security, Compliance & GFR Alignment

10.1 Security Controls

- Authentication & RBAC: JWT-based auth with three roles (Officer, Auditor, Admin), each scoped to the endpoints and data it needs — officers cannot alter audit certificates once issued.
- Transport security: TLS everywhere between client, API gateway and internal services; secrets (DB credentials, DaaS API keys) held in a secrets manager, never in source control.
- Input validation: all comparison requests validate the GeM URL against an allow-listed domain pattern before any scraping job is enqueued, preventing the scraper fleet from being misused as an open proxy.
- Audit logging: every comparison request, certificate issuance and verification check is logged with actor, timestamp and outcome for departmental review.
- Least-privilege scraping: the scraper fleet only ever reads public product pages; it never authenticates as a marketplace user or accesses non-public data.

10.2 GFR 2017, Rule 149 — “Reasonableness of Rates” Alignment

Rule 149 of the General Financial Rules, 2017 requires the procuring authority to record satisfaction that the rate paid is reasonable, having regard to prevailing market conditions. GeM-Intel's audit certificate is designed to be attached directly to the procurement file as this documentary evidence:

- It records the exact comparator listings, landed-cost calculation and Fair Market Value used to reach the compliance verdict.
- It timestamps the check to the moment of purchase decision, not a later reconstruction.
- Its blockchain-hashed content makes it independently verifiable by an auditor without trusting the issuing department's own records.

10.3 Tamper-Proofing via Hashing

Each certificate's canonical content (comparison data + verdict + timestamp) is serialized deterministically and hashed with SHA-256. The hash is anchored on an append-only ledger (a permissioned blockchain, or at minimum a hash-chained log replicated across independent nodes), and the certificate PDF embeds both the hash and a QR code pointing to the public verification endpoint. Any post-issuance edit to the certificate changes its hash and immediately fails verification.

11. Scalability, Performance & Deployment

11.1 Scalability Approach

- Horizontal scaling: the Express API layer and FastAPI AI services are stateless and containerized, so additional instances can be added behind a load balancer as traffic grows.
- Asynchronous workload isolation: scraping and NLP inference — the most resource- and time-intensive steps — run as BullMQ workers that can be scaled independently of the request-handling API tier.
- Caching: Redis absorbs repeat-lookup traffic so the scraper fleet is only invoked for genuinely new or stale products, keeping third-party DaaS costs proportional to unique demand rather than total traffic.
- Database scaling: MongoDB can be sharded by category or department if a single replica set's throughput becomes a bottleneck.

11.2 Suggested Deployment Architecture

- Containerization: every service (React build served via Nginx/CDN, Express API, FastAPI AI service, BullMQ workers, PDF microservice) is packaged as a Docker image.
- Orchestration: Docker Compose is sufficient for hackathon/demo scale; Kubernetes (with Horizontal Pod Autoscaling) is the natural next step for departmental-scale production rollout.
- Environments: separate dev / staging / production environments with environment-specific secrets and DaaS quota limits.
- CI/CD: a standard pipeline (lint → test → build image → deploy) gates every merge to the main branch, with the AI service's matching-accuracy test suite run before deployment.
- Observability: structured logs, request tracing across the Node↔Python boundary, and dashboards for scrape-success rate, queue depth, matching confidence distribution and certificate-issuance volume.

12. Feasibility Analysis & Risk Mitigation

12.1 Feasibility Self-Assessment

Category	Feasibility (1–5)	Rationale
Technical	5.0	Mature open-source stack (MERN, Python/FastAPI, Sentence-Transformers, Playwright, BullMQ/Redis) — buildable within a hackathon-to-production timeline.
Operational	4.0	Slots into a procurement officer's existing workflow; no changes needed to GeM's own systems.
Data	3.5	Hybrid scraping + DaaS APIs keep external collection reliable without depending on unavailable official APIs.
Scalability	4.5	Async job queueing (BullMQ/Redis) + caching scale the workload horizontally as usage grows.
Cost & Deployment	4.0	Cloud-friendly open-source services keep infra cost proportional to usage.

12.2 Risks & Mitigation Strategies

Risk	Mitigation Strategy
Anti-scraping defenses / IP blocking	Commercial DaaS providers (Bright Data, ScrapingBee, SerpApi) for proxy rotation and CAPTCHA handling.
Official GeM/marketplace APIs gated to verified departments/partners	Parse the public catalog now; plug in official APIs once departmental/partner verification is granted.
Semantic-matching false positives on bundled or variant listings	Tune similarity thresholds; route low-confidence matches to manual officer review.
Price volatility / stale cache	Redis caching with scheduled re-scraping keeps price data fresh.
Third-party DaaS cost scaling with volume	Cache-aside design limits DaaS calls to genuinely new/stale products; usage-based budgeting and quota alerts.
Legal/ToS considerations of scraping comparator platforms	Restrict scraping to public product pages, respect robots.txt where applicable, and prefer official/partner APIs as they become available.

13. Impact & Benefits

13.1 Benefits by Category

- **Economic impact —** helps prevent overspending of public funds by directly curbing price gouging and cartelization on GeM listings, protecting taxpayer money at scale.
- **Governance and transparency —** the blockchain-hashed audit certificate provides a tamper-proof, independently verifiable compliance record, strengthening institutional accountability in public procurement.
- **Operational efficiency —** automated TCO normalization and AI-driven semantic matching save significant officer time compared to manual cross-platform price comparison, accelerating procurement cycles.
- **Social trust —** by making price comparisons fair, transparent and auditable, the platform strengthens public confidence in the integrity of government procurement processes.

13.2 Impact by Stakeholder

Stakeholder	Impact
Procurement Officers	One-click, audit-ready proof of “Reasonableness of Rates” — satisfies GFR requirements without manual paperwork.
Departments & PSUs	Automated, apples-to-apples landed-cost comparison in real time at the point of purchase.
Suppliers & GeM	Overpriced or cartelized listings are flagged before a purchase order is raised — encourages fair listings.
Auditors	A tamper-proof, independently verifiable digital trail that reduces effort in post-purchase scrutiny.

14. Implementation Roadmap

Phase	Duration (indicative)	Deliverables
Phase 1 — Foundations	Weeks 1–2	Repo setup, Docker Compose environment, MongoDB/Redis schemas, JWT auth + RBAC, GeM public-catalog parser.
Phase 2 — Data Ingestion	Weeks 3–4	Scraper fleet (Playwright/Selenium) for Amazon/Flipkart/IndiaMART; DaaS integration for proxy rotation; scheduled re-scrape jobs.
Phase 3 — Intelligence Core	Weeks 5–6	Semantic matching pipeline (TF-IDF pre-filter + Sentence-Transformers re-ranking), TCO Normalizer, initial ARIMA/Prophet forecasting.
Phase 4 — Compliance & UX	Weeks 7–8	PDF audit-certificate microservice, blockchain/hash-chain anchoring, verification endpoint, React dashboard, Chrome extension MVP.
Phase 5 — Hardening & Pilot	Weeks 9–10	Load testing, matching-accuracy validation against labeled data, security review, pilot rollout with a small set of procurement officers.

15. Future Scope

- Direct integration with official GeM and marketplace APIs once partner/departmental verification is obtained, reducing reliance on scraping.
- Extending semantic matching with a fine-tuned, domain-specific embedding model trained on government procurement categories for higher accuracy on niche items.
- A supplier-facing view that lets sellers see (without gaming) how their listing compares to market rates, encouraging fair pricing pre-emptively.
- Department-level analytics dashboards showing aggregate savings, flagged-anomaly trends and compliance-certificate volume over time.
- Mobile app for field/on-site procurement officers, mirroring the Chrome extension's one-tap compare workflow.

16. Research & References

- Government e-Marketplace (GeM) portal — https://gem.gov.in — the primary public procurement platform whose product catalog and pricing data form the baseline for all comparisons.
- General Financial Rules (GFR), 2017, Rule 149 — “Reasonableness of Rates” provision — the government mandate requiring documented proof of competitive market pricing before procurement approval, which the audit certificate module is designed to satisfy.
- HuggingFace Sentence-Transformers documentation — https://www.sbert.net — the NLP library underlying the semantic, spec-based product-matching engine.
- ARIMA and Prophet time-series forecasting libraries — the statistical forecasting models used for the predictive price-intelligence and anomaly-detection module.
- Playwright and Puppeteer headless-browser automation documentation — the frameworks powering the multi-source web scraper fleet and the PDF audit-certificate generation microservice.
- Data-as-a-Service (DaaS) providers — Bright Data, ScrapingBee, SerpApi — commercial services used for proxy rotation and CAPTCHA handling to sustain reliable, large-scale marketplace data collection.